

My PAI — Personal AI Assistant: Project Documentation

Author: Kristian Garchev

Table of Contents

- 1. [Project Overview](#)
- 2. [Functionality](#)
- 3. [Architecture](#)
- 4. [Implementation Details](#)
- 5. [Data Model](#)
- 6. [Datasets Used](#)
- 7. [Configuration](#)
- 8. [Technologies and Libraries Used](#)
- 9. [Problems and Solutions](#)
- 10. [References](#)
- 11. [Appendix: Requirements Traceability](#)

1. Project Overview

My PAI (Personal AI Assistant) is a full-stack, self-hosted web application that provides a ChatGPT-like conversational interface combined with a personal file workspace. The system is designed as a multi-agent microservices architecture, deployed via Kubernetes (or Docker Compose for local development), and runs entirely on a single machine equipped with an NVIDIA GPU.

Key Characteristics

- **Single-user system:** A hardcoded user (user_id = 1) is always authenticated; there is no login/logout flow.
- **ChatGPT clone with enhanced file handling:** Beyond chat, the system integrates file management, semantic search, voice I/O, web search, Spotify playback, file format transformations, and general-purpose computation.
- **Local-first AI:** All LLM inference is performed locally using Ollama (models such as LLaMA 3.2 and LLaVA 7B), with no data sent to third-party cloud AI providers.
- **Microservices architecture:** The backend is decomposed into 11 agents and 5 services, each running as an independent FastAPI application within its own container/pod.
- **Target hardware:** Windows laptop with WSL2, NVIDIA GeForce RTX 3070 (8 GB VRAM), ~32 GB system RAM.

2. Functionality

2.1 Chat Interface

The primary user-facing feature is a conversational chat interface. Users can:

- Create, list, rename, and delete chat sessions.
- Send natural-language messages and receive AI-generated responses.
- Reference workspace files within messages (e.g., `<file path="/docs/report.pdf"/>`).
- View responses rendered in Markdown, with syntax-highlighted code blocks.
- Responses are immutable once created (append-only message log).

The chat flow internally proceeds through:

1. The **Orchestrator Agent** receives the user message.
2. The **Task Planner Agent** analyzes intent and produces an execution plan (a JSON DSL).
3. The Orchestrator executes each plan step by calling the relevant agent(s) sequentially.
4. The **Conversational Agent** generates the final natural-language response using RAG context.
5. The response is stored via the **ChatStore Agent** and returned to the frontend.

2.2 Workspace (File System) Management

Users have a personal workspace (virtual file system) where they can:

Operation	Description
List	Browse files and subdirectories in any directory
Upload	Upload files (stored in MinIO, metadata in PostgreSQL/MySQL)
Download / Read	Retrieve file content by path
	Copy files or entire directories (recursive), creating new MinIO

Copy Operation	objects Description
Move	Move files or directories (updates metadata paths, no MinIO copy)
Delete	Remove files or directories recursively
Rename	Rename a file or directory
Write	Create/overwrite a text file programmatically

File uploads automatically trigger re-indexing via the SDSA Agent, which chunks documents, generates embeddings, and stores them in ChromaDB for semantic search.

2.3 Semantic Document Search (SDSA)

The Semantic Document Store Agent (SDSA) provides:

- **Document chunking:** Text files are split into overlapping chunks; binary files (PDF, DOCX, PPTX) are parsed and chunked; images are captioned and OCR'd.
- **Text embeddings:** Generated using `sentence-transformers/all-MiniLM-L6-v2`.
- **Image embeddings:** Generated using `openai/clip-vit-base-patch32` (CLIP model).
- **Image captioning:** via LLaVA 7B model through Ollama.
- **OCR:** Tesseract OCR for text extraction from images.
- **Text summarization:** via Ollama (LLaMA 3.2:3b).
- **Similarity search:** Cosine similarity queries against ChromaDB, returning the top-k most relevant chunks.
- **Message indexing:** Chat messages are also embedded and indexed, enabling semantic search across conversation history.

2.4 Web Search

The Web Search Agent provides:

- **DuckDuckGo search:** Keyword-based web search (no API key required), scraping the lite HTML version.
- **URL content fetching:** Retrieves and extracts main text content from web pages, stripping navigation, scripts, and boilerplate.
- **Search-and-fetch:** Combined search + content retrieval for top results.
- **Image search & download:** Fetches images from the web and returns them as base64.

2.5 Spotify Integration

The Spotify Agent provides music playback control:

- **Authentication:** OAuth2 via stored client credentials and refresh tokens (persisted in APIKeys Service).
- **Playback control:** Play, pause, next track, previous track, set volume, shuffle.
- **Search:** Search Spotify's catalog for tracks.
- **Play by query:** Search and immediately play the first result.

2.6 Voice I/O

The Voice I/O Agent handles:

- **Speech-to-Text (STT):** Uses the Whisper model via `faster-whisper`. Supports file upload and base64-encoded audio. Accepts WAV, MP3, M4A, OGG, and FLAC formats. Uses GPU (CUDA) with float16, falling back to CPU with int8.
- **Text-to-Speech (TTS):** Uses `pyttsx3` as the synthesis engine (Coqui TTS was removed due to dependency conflicts). Returns base64-encoded WAV audio.

2.7 File Transformations

The File Transform Agent supports:

Conversion	Method
Markdown → HTML	<code>markdown</code> library with extensions
Markdown → PDF	Markdown → HTML → PDF (via WeasyPrint/pdfkit)
HTML → PDF	WeasyPrint or pdfkit
HTML → Markdown	<code>html2text</code>
Image format conversion (PNG ↔ JPG ↔ WebP ↔ BMP ↔ GIF)	Pillow (PIL)
PDF → Text	PyPDF2
PDF → Images	PyMuPDF (fitz)

Text → PDF Conversion	Text → HTML → PDF Method
Image resize	Pillow
Text extraction (OCR)	Tesseract via pytesseract

2.8 General Task Agent

A versatile computation agent offering:

- **Math evaluation:** Safe evaluation of mathematical expressions using a restricted set of allowed functions (`math` module).
- **Code execution:** Sandboxed Python execution with restricted builtins (`no os , sys , subprocess , open , etc.`).
- **Data analysis:** Statistical operations (mean, median, mode, std, describe) on numeric datasets.
- **Unit conversion:** Length, mass, time, data size, and temperature conversions (using `pint` library and custom mappings).
- **Random number generation.**

2.9 Task Planning

The Task Planner Agent uses an LLM (LLaMA 3.2) to:

1. Analyze user intent from the message and conversation context.
2. Detect which agents are needed based on trigger-word matching and LLM analysis.
3. Generate a structured execution plan as a JSON DSL — an ordered list of tasks specifying agent, action, parameters, and inter-task dependencies.
4. Support parameter references between tasks (e.g., `${t1.result.content}`).

2.10 Orchestration

The Orchestrator Agent is the single entry point for the frontend. It:

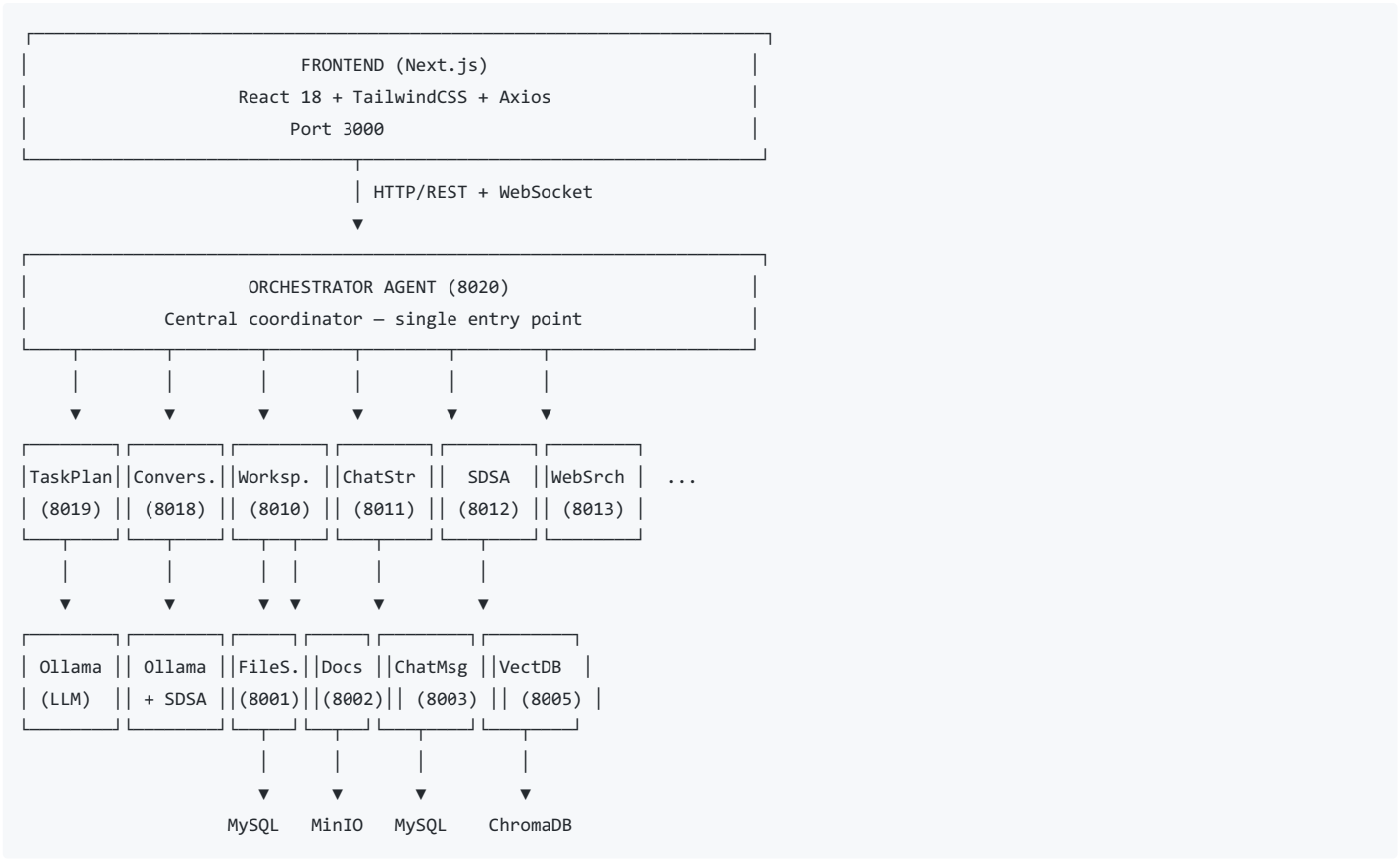
1. Receives user messages via REST (`POST /chat`) or WebSocket for streaming.
2. Stores the user message in chat history (via ChatStore Agent).
3. Sends the message to the Task Planner for a structured plan.
4. Executes plan steps **sequentially** (to avoid concurrent GPU usage).
5. Collects results from each agent call, resolving parameter dependencies.
6. Calls the Conversational Agent to generate the final Markdown response using all gathered context.
7. Stores the system response and returns it to the frontend.
8. Supports file uploads alongside messages (multipart form data).

3. Architecture

3.1 High-Level Architecture

The system follows a **two-tier microservices architecture**:

- **Data Layer (Services):** Five services that own and manage persistent data stores. These are private and accessible only to their respective agents.
- **Application Layer (Agents):** Eleven agents that implement business logic, communicate with services, and interact with ML models.



3.2 Service-Port Mapping

Data Layer Services

Service	Port	Data Store	Purpose
FileSystem Service	8001	MySQL (PostgreSQL per spec)	File path metadata, directory structure
Documents Service	8002	MinIO	Binary file content (S3-compatible object store)
ChatMessages Service	8003	MySQL	Chat sessions and messages
APIKeys Service	8004	MySQL	API keys (Spotify credentials)
VectorDB Service	8005	ChromaDB	Vector embeddings and semantic search

Application Layer Agents

Agent	Port	GPU Required	Dependencies
Workspace Agent	8010	No	FileSystem Svc, Documents Svc, SDSA Agent
ChatStore Agent	8011	No	ChatMessages Svc, SDSA Agent
SDSA Agent	8012	Yes	VectorDB Svc, Documents Svc, Ollama
WebSearch Agent	8013	No	External web (DuckDuckGo)
Spotify Agent	8014	No	APIKeys Svc, Spotify Web API
VoiceIO Agent	8015	Yes	Whisper model (GPU), pyttsx3

Agent	Port	GPU Required	Dependencies
Transform Agent	8016	No	Workspace Agent
GeneralTask Agent	8017	No	(Self-contained)
Conversational Agent	8018	Yes (via Ollama)	Ollama, SDSA Agent, ChatStore Agent
TaskPlanner Agent	8019	Yes (via Ollama)	Ollama
Orchestrator Agent	8020	No	All other agents

3.3 Communication Patterns

- **All inter-service communication is synchronous HTTP/REST+JSON.** This design decision was made to prevent concurrent GPU usage that would exhaust the 8 GB VRAM.
- The Orchestrator executes plan steps **sequentially**, never in parallel, ensuring only one ML model is active at a time.
- **Frontend → Backend:** The frontend communicates exclusively with the Orchestrator Agent via REST (`POST /chat`) or WebSocket for streaming responses.
- **Agents → Services:** Agents call their backing services via internal HTTP REST endpoints. Services are private and not directly exposed externally.

3.4 Deployment Architecture

Docker Compose (Local Development)

All containers run on a single Docker network (`pai-network`). Docker Compose defines:

- 4 infrastructure containers: MySQL, MinIO, ChromaDB, Ollama
- 5 service containers
- 11 agent containers
- 1 frontend container

GPU-intensive containers (SDSA Agent, VoiceIO Agent, Ollama) use NVIDIA runtime with `nvidia.com/gpu` reservations.

Kubernetes (Production / WSL2)

The Kubernetes deployment is split across four manifest files:

File	Contents
<code>01-infrastructure.yaml</code>	Namespace, ConfigMap, Secrets, PVCs, MySQL, MinIO, ChromaDB, Ollama deployments + services
<code>02-services.yaml</code>	All 5 data-layer service deployments + ClusterIP services
<code>03-agents.yaml</code>	All 11 agent deployments + ClusterIP services
<code>04-frontend.yaml</code>	Frontend deployment, service, and Ingress

PersistentVolumeClaims (PVCs):

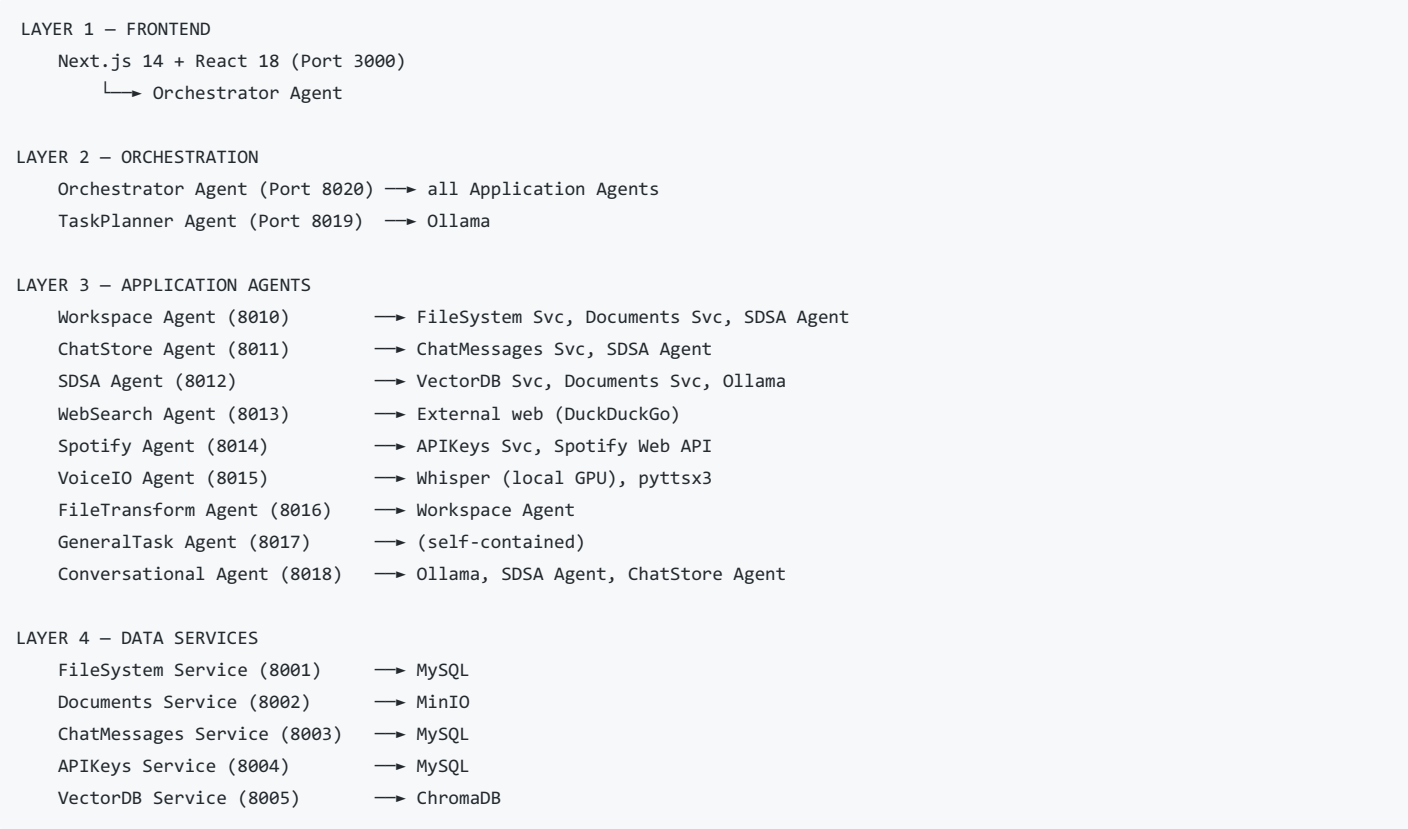
PVC	Size	Used By
<code>mysql-pvc</code>	10 Gi	MySQL data
<code>minio-pvc</code>	50 Gi	MinIO objects
<code>chromadb-pvc</code>	20 Gi	ChromaDB embeddings
<code>ollama-pvc</code>	50 Gi	Ollama model files
<code>model-cache-pvc</code>	(shared)	Sentence-transformers, CLIP caches

Resource allocations:

Pod Type	CPU Req	Mem Req	GPU
Data services	50m	128 Mi	—
Non-GPU			

agents Pod Type	50m–100m CPU Req	128–512 Mi Mem Req	— GPU
SDSA Agent	500m	4 Gi	1× NVIDIA
VoiceIO Agent	500m	4 Gi	1× NVIDIA
Conversational / TaskPlanner	500m	4 Gi	1× NVIDIA (via Ollama)
Ollama	1000m	8 Gi	1× NVIDIA

3.5 System Architecture Diagram



4. Implementation Details

4.1 Backend – Python + FastAPI

Each backend service and agent is implemented as a standalone FastAPI application, served by Uvicorn. Key implementation patterns:

4.1.1 Application Structure (per Agent/Service)

Each agent/service follows a consistent structure:

```
agent_name/  
├─ __init__.py      # Empty or re-exports  
├─ main.py          # FastAPI app, endpoints, business logic
```

The SDSA Agent has additional helper modules:

```
sdsa_agent/
├── __init__.py
├── main.py          # FastAPI app, indexing/query endpoints
├── chunkers.py      # Document chunking (TextChunker, PDFChunker, etc.)
├── chunk_embedders.py # Embedding generation (TextChunkEmbedder, ImageChunkEmbedder)
├── text_embedder.py  # sentence-transformers wrapper (lazy-load / unload)
├── image_embedder.py # CLIP model wrapper (lazy-load / unload)
├── image_captioner.py # LLaVA captioning via Ollama
├── image_text_extractor.py # Tesseract OCR
└── text_summarizer.py # LLM-based text summarization via Ollama
```

4.1.2 Shared Code

The `backend/shared/` directory contains:

- **config.py**: Centralized Settings dataclass loading service URLs, database configurations, and agent URLs from environment variables.
- **models.py**: Pydantic models shared across agents/services — ChatCreate, MessageResponse, FileRecord, EmbeddingRecord, ChunkMetadata, etc.
- **utils.py**: Common utility functions.

4.1.3 Database Layer (SQLAlchemy + MySQL)

Services that use MySQL (FileSystem, ChatMessages, APIKeys) use SQLAlchemy ORM:

- Each service has its own `database.py` (engine, session factory, `init_db()` function) and `models.py` (SQLAlchemy table definitions).
- Database initialization happens automatically on startup (`@app.on_event("startup")`), creating tables if they don't exist.
- Connection strings are assembled from environment variables (host, port, user, password, database).

4.1.4 GPU Memory Management

A critical design pattern used by GPU-intensive components is **lazy loading and explicit unloading** of ML models:

```
_model = None

def get_model():
    """Lazy load model only when needed."""
    global _model
    if _model is None:
        _model = SentenceTransformer("all-MiniLM-L6-v2")
    return _model

def unload_model():
    """Free GPU memory after use."""
    global _model
    if _model is not None:
        del _model
        _model = None
        import gc; gc.collect()
        torch.cuda.empty_cache()
```

This pattern is used for:

- Text embedder (sentence-transformers)
- Image embedder (CLIP)
- Whisper (STT)
- All Ollama-based models are managed by the Ollama server itself

After every embedding/transcription operation, models are explicitly unloaded to free VRAM for subsequent operations.

4.1.5 Task Planner DSL

The Task Planner generates JSON execution plans following this schema:

```

{
  "plan_id": "plan_abc123",
  "user_request": "Find an image of a sunset and save it to /images/",
  "intent": "web_image_download_and_save",
  "tasks": [
    {
      "task_id": "t1",
      "description": "Search for sunset image online",
      "agent": "websearch",
      "action": "download_image",
      "parameters": {"query": "beautiful sunset"},
      "depends_on": [],
      "priority": 1
    },
    {
      "task_id": "t2",
      "description": "Save the downloaded image to workspace",
      "agent": "workspace",
      "action": "save_image",
      "parameters": {
        "path": "/images/sunset.jpg",
        "image_base64": "${t1.result.image_base64}",
        "content_type": "image/jpeg"
      },
      "depends_on": ["t1"],
      "priority": 2
    }
  ],
  "parallel_groups": [["t1"], ["t2"]]
}

```

Parameter references (`${t1.result.image_base64}`) enable data flow between task steps.

4.1.6 Orchestrator Action Mapping

The Orchestrator maintains a mapping of agent names → actions → HTTP endpoints:

```

AGENT_ACTIONS = {
  "workspace": {
    "list_files": ("GET", "/list", lambda p: {"directory": p.get("path", "/")}),
    "read_file": ("GET", "/file/text", lambda p: {"path": p.get("path")}),
    "write_file": ("POST", "/write", lambda p: {...}),
    ...
  },
  "websearch": {
    "search_web": ("GET", "/search", lambda p: {...}),
    "download_image": ("POST", "/images/download", lambda p: {...}),
    ...
  },
  ...
}

```

This allows the Orchestrator to dynamically route any plan step to the correct agent endpoint.

4.2 Frontend — Next.js + React

4.2.1 Technology Stack

- **Next.js 14** with server-side rendering (SSR) for fast initial page loads
- **React 18** with functional components and hooks
- **TypeScript** for type safety
- **TailwindCSS** for utility-first styling
- **Axios** for HTTP communication with the Orchestrator

4.2.2 Page Structure

Route	Component	Purpose
/	index.tsx	Home page / redirect to chat
/chat	chat/	Chat interface
/workspace	workspace/	File browser and manager
/settings	settings/	Application settings (Spotify keys, etc.)

4.2.3 Component Hierarchy

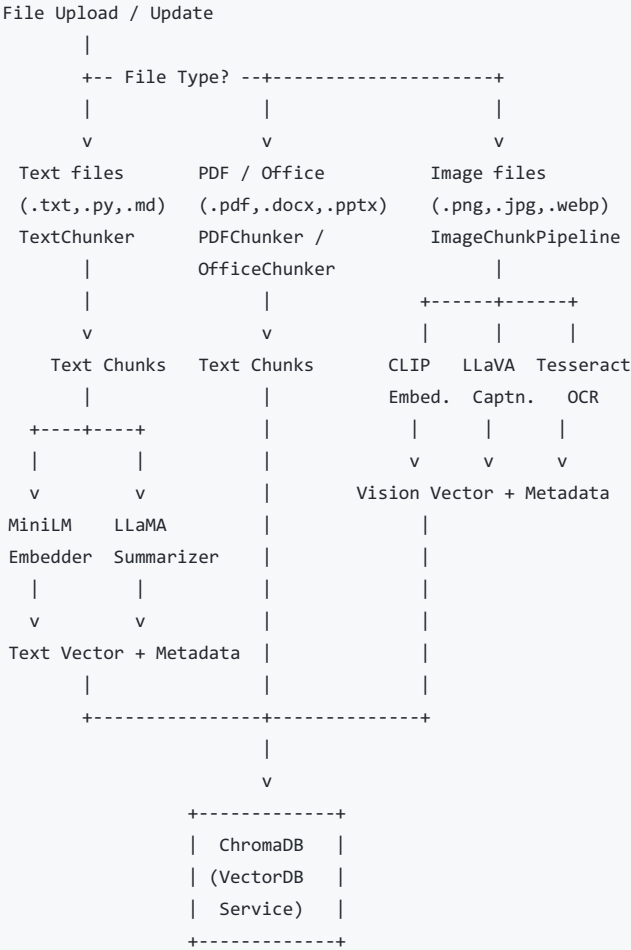
Layout/	
├─ Sidebar.tsx	# Navigation sidebar (chats list, workspace link)
Chat/	
├─ ChatList.tsx	# List of chat sessions
├─ ChatMessage.tsx	# Individual message rendering (Markdown, code)
├─ ChatInput.tsx	# Message input with file attachment support
Workspace/	
├─ FileBrowser.tsx	# Directory tree and file listing
├─ FilePreview.tsx	# File content preview with syntax highlighting

4.2.4 Key Frontend Libraries

Library	Purpose
react-markdown	Render Markdown in chat messages
react-syntax-highlighter	Syntax highlighting for code blocks
remark-gfm	GitHub Flavored Markdown support
@heroicons/react	Icon library
mammoth	DOCX preview in browser

4.3 SDSA Agent – Processing Pipeline

The SDSA (Semantic Document Store Agent) is the most complex agent. Its document processing pipeline works as follows:



Dual embedding for images: Each image produces **two** embedding records:

1. A **vision embedding** (from CLIP) for visual similarity search.
2. A **text embedding** (from sentence-transformers) of the concatenated caption and OCR text, for textual similarity search.

RAG (Retrieval-Augmented Generation): When the Conversational Agent receives a query:

1. It sends the query to SDSA for semantic search.
2. SDSA embeds the query text and queries ChromaDB for the top-k nearest neighbors.
3. Matching document/message chunks are returned with metadata.
4. The Conversational Agent formats these as context and prepends them to the LLM prompt.
5. For image chunks, the actual image data is fetched and included as base64 in the LLaVA model call.

5. Data Model

5.1 Relational Database Schema (MySQL)

The system uses MySQL 8 (InnoDB) to store relational data across three functional domains. The database schema is implemented via SQLAlchemy ORM models.

5.1.1 Entity-Relationship Diagram

+-----+		+-----+		+-----+	
USER		CHAT		CHAT_MESSAGE	
-----		-----		-----	
user_id	--1:N--	user_id (FK)		chat_id (FK)	
(PK)		chat_id (PK)	--1:N--	message_id (PK)	
		chat_name		created_at	
		created_at		message_text	
+---+---+		+-----+		user_role	
				+-----+	
		+-----+			
+---1:N----		FILE			

		user_id (FK)			
		filepath (PK)			
		uuid			
		file_type			
		created_at			
		updated_at			
		+-----+			
		+-----+			
+---1:N----		SUBDIRECTORY			

		user_id (FK,PK)			
		directory (PK)			
		subdirectory (PK)			
		created_at			
		updated_at			
		+-----+			
		+-----+			
+---1:1----		API_KEY			

		user_id (PK)			
		spotify_api_key			
		spotify_client_id			
		spotify_client_			
		secret			
		spotify_redirect_			
		uri			
		created_at			
		updated_at			
		+-----+			

5.1.2 Table Details

CHATS

Column	Type	Constraints	Description
user_id	INT	PK (composite), FK→USERS	Owner user
chat_id	INT	PK (composite), auto-increment per user	Unique chat ID
chat_name	VARCHAR(255)	NOT NULL	Display name
created_at	DATETIME	NOT NULL	Creation timestamp

CHAT_MESSAGES

Column	Type	Constraints	Description

chat_id Column	INT Type	FK→CHATS.chat_id Constraints	Parent chat Description
message_id	INT	Auto-increment per chat	Message sequence
created_at	DATETIME	NOT NULL	Timestamp (UTC)
message_text	TEXT	NOT NULL	Message content (immutable)
user_role	ENUM('user','assistant','system')	NOT NULL	Sender role

Primary Key: (chat_id, message_id) . Index on (chat_id) for fast message retrieval.

FILES

Column	Type	Constraints	Description
user_id	INT	PK (composite)	Owner
filepath	VARCHAR(1024)	PK (composite)	Full workspace path (e.g., /docs/report.pdf)
uuid	CHAR(36)	NOT NULL, UNIQUE(user_id, uuid)	UUID of content in MinIO
file_type	VARCHAR(50)	NOT NULL	File extension (e.g., pdf , png)
created_at	DATETIME	NOT NULL	Upload time
updated_at	DATETIME	NOT NULL	Last modification

Primary Key: (user_id, filepath) . Index on (user_id) for directory listing.

SUBDIRECTORIES

Column	Type	Constraints	Description
user_id	INT	PK (composite)	Owner
directory	VARCHAR(1024)	PK (composite)	Parent directory path
subdirectory	VARCHAR(255)	PK (composite)	Child directory name
created_at	DATETIME	NOT NULL	Creation time
updated_at	DATETIME	NOT NULL	Modification time

Primary Key: (user_id, directory, subdirectory) . Index on (user_id, directory) .

API_KEYS

Column	Type	Constraints	Description
user_id	INT	PK	Owner
spotify_api_key	VARCHAR(255)	NULL	Spotify refresh token
spotify_client_id	VARCHAR(255)	NULL	Spotify app client ID
spotify_client_secret	VARCHAR(255)	NULL	Spotify app client secret
spotify_redirect_uri	VARCHAR(255)	NULL	OAuth redirect URI
created_at	DATETIME	NOT NULL	Creation time
updated_at	DATETIME	NOT NULL	Modification time

5.2 Object Storage (MinIO)

MinIO stores raw file content in an S3-compatible object store:

- **Bucket:** documents
- **Object key:** UUID (matching the `uuid` column in the FILES table)
- **Metadata:** Content-Type header is preserved
- **Operations:** PUT (upload), GET (download), DELETE, COPY, STAT

Files are never accessed directly by path in MinIO — the indirection via UUID enables efficient renaming and moving without actual data copy.

5.3 Vector Database (ChromaDB)

ChromaDB stores vector embeddings for semantic search:

- **Collection:** documents (cosine similarity metric)
- **Record structure:**

Field	Type	Description
chunk_id	String (ID)	Format: {user_id}:{source_type}: {source_id}:{chunk_type}: {chunk_index}
embedding	Float vector	384-dim (text, MiniLM) or 512-dim (image, CLIP)
document	String	Chunk text content, or "user_id::filepath" for images
metadata	Dict	user_id, source_type, source_id, chunk_type, chunk_index, summary, caption, extracted_text, embedding_type

The metadata enables filtered queries (e.g., search only documents vs. messages, filter by user).

6. Datasets Used

6.1 User-Generated Data

The system does not come with pre-loaded datasets. All data is user-generated:

- **Chat messages:** Created by user interactions.
- **Workspace files:** Uploaded by the user (documents, images, code, etc.).
- **Embeddings:** Automatically generated from uploaded files and chat messages.

6.2 Pre-trained Models (External Datasets)

The following pre-trained models are used, each trained on different datasets:

Model	Training Data	Purpose
all-MiniLM-L6-v2	1B+ sentence pairs from multiple datasets	Text embeddings (384-dim)
clip-vit-base-patch32	400M image-text pairs (WebImageText)	Image embeddings (512-dim)
llama3.2 (via Ollama)	Undisclosed (Meta's training data)	Chat, summarization, task planning
llava:7b (via Ollama)	LLaVA instruction-tuning data + visual	Image captioning, multimodal chat
faster-whisper	680,000 hours of	

(base) Model	multilingual audio Training Data	Speech-to-text Purpose
-----------------	-------------------------------------	---------------------------

6.3 Example Datasets (Provided)

The project includes example datasets for testing and demonstration:

- `backend/examples/homework_example/` : Sample homework assignment with RAG chat application — demonstrates document indexing and question-answering.
- `backend/examples/multi_modal_embedding_example/` : Example of multi-modal embedding generation.
- `backend/examples/Prompt-Engineering-Guide-main/` : A full clone of the Prompt Engineering Guide — useful for testing document ingestion and semantic search on a large corpus.
- `backend/examples/simple_example/` : Simple RAG application notebook and script.

7. Configuration

7.1 Environment Variables

All services are configured via environment variables, with sensible defaults for local development. The `backend/shared/config.py` `Settings` class centralizes all URL configurations.

Infrastructure

Variable	Default	Description
<code>MYSQL_HOST</code>	<code>mysql-service</code>	MySQL server hostname
<code>MYSQL_PORT</code>	<code>3306</code>	MySQL port
<code>MYSQL_USER</code>	<code>pai_user</code>	MySQL username
<code>MYSQL_PASSWORD</code>	<code>pai_password</code>	MySQL password
<code>MYSQL_DATABASE</code>	<code>pai_db</code>	MySQL database name
<code>MINIO_ENDPOINT</code>	<code>minio:9000</code>	MinIO S3 endpoint
<code>MINIO_ACCESS_KEY</code>	<code>minioadmin</code>	MinIO access key
<code>MINIO_SECRET_KEY</code>	<code>minioadmin</code>	MinIO secret key
<code>MINIO_BUCKET</code>	<code>documents</code>	MinIO bucket name
<code>CHROMADB_HOST</code>	<code>chromadb-service</code>	ChromaDB host
<code>CHROMADB_PORT</code>	<code>8000</code>	ChromaDB port
<code>OLLAMA_URL</code>	<code>http://ollama:11434</code>	Ollama LLM server URL

Service URLs

Variable	Default	Description
<code>FILESYSTEM_SERVICE_URL</code>	<code>http://filesystem-service:8001</code>	FileSystem Service
<code>DOCUMENTS_SERVICE_URL</code>	<code>http://documents-service:8002</code>	Documents Service
<code>CHATMESSAGES_SERVICE_URL</code>	<code>http://chatmessages-service:8003</code>	ChatMessages Service
<code>APIKEYS_SERVICE_URL</code>	<code>http://apikeys-service:8004</code>	APIKeys Service
<code>VECTORDB_SERVICE_URL</code>	<code>http://vectordb-service:8005</code>	VectorDB Service

Agent URLs

Variable	Default	Description
WORKSPACE_AGENT_URL	http://workspace-agent:8010	Workspace Agent
CHATSTORE_AGENT_URL	http://chatstore-agent:8011	ChatStore Agent
SDSA_AGENT_URL	http://sdsa-agent:8012	SDSA Agent
WEBSEARCH_AGENT_URL	http://websearch-agent:8013	WebSearch Agent
SPOTIFY_AGENT_URL	http://spotify-agent:8014	Spotify Agent
VOICEIO_AGENT_URL	http://voiceio-agent:8015	VoicelO Agent
FILETRANSFORM_AGENT_URL	http://filetransform-agent:8016	FileTransform Agent
GENERALTASK_AGENT_URL	http://generaltask-agent:8017	GeneralTask Agent
CONVERSATIONAL_AGENT_URL	http://conversational-agent:8018	Conversational Agent
TASKPLANNER_AGENT_URL	http://taskplanner-agent:8019	TaskPlanner Agent
ORCHESTRATOR_URL	http://orchestrator-agent:8020	Orchestrator Agent

ML Models

Variable	Default	Description
TEXT_EMBEDDING_MODEL	sentence-transformers/all-MiniLM-L6-v2	Text embedding model
IMAGE_EMBEDDING_MODEL	openai/clip-vit-base-patch32	Image embedding model
IMAGE_CAPTION_MODEL	llava:7b	Image captioning model
SUMMARIZER_MODEL	llama3.2:3b	Text summarization model
DEFAULT_LLM_MODEL	llava:7b	Default chat LLM (multimodal)
PLANNER_MODEL	llama3.2	Task planner LLM
WHISPER_MODEL	base	Whisper STT model size

7.2 Kubernetes Configuration

Kubernetes configuration is stored in `kubernetes/01-infrastructure.yaml`:

- **ConfigMap** (`pai-config`): All non-secret environment variables.
- **Secret** (`pai-secrets`): Database passwords, MinIO keys.
- **PVCs**: Persistent storage claims for MySQL (10 Gi), MinIO (50 Gi), ChromaDB (20 Gi), Ollama models (50 Gi).

7.3 Docker Compose Configuration

The `docker-compose.yaml` defines the complete local development stack with:

- Named volumes for persistence (`mysql_data`, `minio_data`, `chromadb_data`, `ollama_data`, `model_cache`)
- Health checks for MySQL and MinIO
- GPU reservations for Ollama, SDSA Agent, and VoicelO Agent
- A shared network (`pai-network`)

7.4 Conda Environment

The `backend/config.yaml` defines a Conda environment (`my_pai`) with Python 3.10, CUDA 11.2, cuDNN 8.1, and all ML dependencies for local development outside of Docker.

8. Technologies and Libraries Used

8.1 Infrastructure & Storage

Technology	Version	Role
MinIO	Latest	S3-compatible object storage for document/file content
ChromaDB	0.4.x–0.5.x	Vector database storing embeddings with metadata for semantic search
MySQL 8 (InnoDB)	8.0	Relational database for chat history, file metadata, API keys, directory structure
Ollama	Latest	Local LLM inference server (hosts LLaMA 3.2, LLaVA 7B)
Docker / Docker Compose	Latest	Containerization and local orchestration
Kubernetes (k3s / Docker Desktop)	Latest	Production-grade container orchestration on WSL2
NVIDIA CUDA	12.1	GPU acceleration for ML model inference

8.2 Backend — Python

Library	Version	Purpose
FastAPI	≥0.109.0	Async web framework for REST APIs
Uvicorn	≥0.27.0	ASGI server for FastAPI
Pydantic	≥2.5.0	Data validation and serialization
httpx	≥0.25.0	Async HTTP client for inter-service calls
SQLAlchemy	≥2.0.0	ORM for MySQL database access
PyMySQL	≥1.1.0	MySQL driver for SQLAlchemy
minio (Python client)	≥7.2.0	MinIO S3 client for document storage
chromadb	≥0.4.0	ChromaDB Python client
ollama (Python client)	≥0.1.6	Ollama API client
sentence-transformers	≥2.2.0	Text embedding generation (all-MiniLM-L6-v2)
transformers	≥4.36.0	CLIP model hosting
torch	≥2.1.0	PyTorch for ML inference
torchvision	≥0.16.0	Image processing for CLIP
open-clip-torch	≥2.20.0	CLIP model variants
Pillow	≥10.0.0	Image processing and format conversion

faster-whisper Library	Version	Purpose (Whisper STT or Translate2-accelerated)
pyttsx3	≥2.90	Text-to-speech (system TTS)
pytesseract	≥0.3.10	OCR via Tesseract
PyPDF2	≥3.0.0	PDF reading and manipulation
pdfplumber	≥0.10.0	PDF text extraction
python-docx	≥1.0.0	DOCX parsing
python-pptx	≥0.6.23	PPTX parsing
olefile	≥0.47	Legacy .ppt/.doc binary format reading
markdown	≥3.5.0	Markdown → HTML conversion
beautifulsoup4	≥4.12.0	HTML parsing and content extraction
duckduckgo-search	≥4.0	DuckDuckGo web search
spotipy	≥2.23.0	Spotify Web API client
pandas	≥2.0.0	Data analysis
numpy	≥1.26.0	Numerical computation
pint	≥0.23	Unit conversion library
nbformat	≥5.9.0	Jupyter notebook parsing
python-dotenv	≥0.21.0	.env file loading
PyYAML	≥6.0.0	YAML configuration parsing
python-multipart	≥0.0.6	File upload handling in FastAPI

8.3 Frontend — JavaScript / TypeScript

Library	Version	Purpose
Next.js	14.0.4	React framework with SSR
React	18.2.x	UI component library
TypeScript	5.3.x	Type-safe JavaScript
TailwindCSS	3.3.x	Utility-first CSS framework
Axios	1.6.x	HTTP client for API calls
react-markdown	9.0.x	Markdown rendering in chat
react-syntax-highlighter	15.5.x	Code syntax highlighting
remark-gfm	4.0.x	GitHub Flavored Markdown
@heroicons/react	2.1.x	SVG icon library
mammoth	1.6.x	DOCX → HTML preview

8.4 DevOps & Build Tools

Tool	Purpose
Docker	Container builds (multi-stage Dockerfiles)
Docker Compose	Local development orchestration
Kubernetes	Production deployment manifests
NGINX Ingress	External access routing
npm	Frontend dependency management
pip	Python dependency management
Conda/Micromamba	ML environment management (local dev)

9. Problems and Solutions

9.1 GPU Memory Constraints (8 GB VRAM)

Problem: Multiple ML models (LLaMA, LLaVA, CLIP, Whisper, sentence-transformers) cannot be loaded simultaneously on an 8 GB GPU.

Solution:

- **Lazy loading:** Models are loaded into GPU memory only when needed and explicitly unloaded (`del model; torch.cuda.empty_cache()`) after each operation.
- **Sequential execution:** The Orchestrator processes plan steps one at a time, never in parallel, ensuring at most one GPU-intensive model is active.
- **Ollama manages LLM lifecycle:** Ollama handles model loading/unloading for LLaMA and LLaVA, preventing manual management of these large models.
- **Kubernetes GPU scheduling:** Pods requiring GPU declare `nvidia.com/gpu: 1` in resource limits, so the Kubernetes scheduler prevents concurrent GPU pods.

9.2 Synchronous Communication Requirement

Problem: Asynchronous microservice communication could result in two GPU models being invoked concurrently, exceeding VRAM.

Solution: All inter-service calls are synchronous HTTP/REST. The Orchestrator waits for each agent call to complete before initiating the next. No message queues or event buses are used.

9.3 Binary File Handling in JSON APIs

Problem: Binary files (PDF, DOCX, images) cannot be directly embedded in JSON request/response bodies.

Solution: Binary content is base64-encoded when passed between services via JSON. The `is_base64` flag in SDSA's `DocumentIndexRequest` distinguishes binary from text content. MinIO directly handles binary uploads/downloads via multipart form data.

9.4 Legacy File Format Support (.ppt, .doc)

Problem: Legacy Microsoft Office binary formats (OLE2) are not supported by modern Python libraries like `python-pptx` or `python-docx`.

Solution: The Workspace Agent includes a custom `extract_text_from_ppt()` function that uses `olefile` to read PowerPoint Document streams, parsing binary record headers (TextCharsAtom 0x0FA0 for Unicode text, TextBytesAtom 0x0FA8 for ASCII) to extract slide text.

9.5 Coqui TTS Dependency Conflicts

Problem: Coqui TTS had dependency conflicts with other ML libraries in the project environment.

Solution: TTS was replaced with `pyttsx3`, a lightweight system-level TTS engine. While lower quality, it avoids dependency issues and does not require GPU.

9.6 DuckDuckGo Search Reliability

Problem: DuckDuckGo has no official public API, and scraping-based approaches are fragile.

Solution: The Web Search Agent scrapes DuckDuckGo's lite HTML version (`https://html.duckduckgo.com/html/`), parsing results with BeautifulSoup. URL extraction handles DuckDuckGo's redirect wrapping (`uddg=` parameter). The system gracefully returns empty results on failure.

9.7 Persistent Data Across Pod Restarts

Problem: Kubernetes pods are ephemeral; data stored in container filesystems is lost on restart.

Solution: All stateful components (MySQL, MinIO, ChromaDB, Ollama models, ML model caches) use PersistentVolumeClaims (PVCs). The Kubernetes manifests

define PVCs ranging from 10 Gi (MySQL) to 50 Gi (MinIO, Ollama).

9.8 File Path Normalization

Problem: Inconsistent file path formats (trailing slashes, missing leading slashes, double slashes) cause lookup failures.

Solution: The Workspace Agent applies consistent path normalization:

```
def normalize_path(path: str) -> str:
    if not path: return "/"
    if not path.startswith("/"): path = "/" + path
    if path != "/" and path.endswith("/"): path = path.rstrip("/")
    return path
```

9.9 Concurrent Model Downloads

Problem: Multiple agents trying to download large models (sentence-transformers, CLIP) simultaneously caused slow startup and disk contention.

Solution: A shared `model_cache` volume is mounted across GPU agents, so models downloaded once are available to all agents.

9.10 LLM Output Parsing

Problem: LLM responses for task planning are not always valid JSON – they may include markdown code fences, conversational preamble, or malformed syntax.

Solution: The Task Planner includes a robust `parse_json_from_llm()` function that:

1. Tries to extract JSON from ````json`` code blocks.
2. Falls back to parsing the entire response as JSON.
3. Searches for JSON object/array patterns using regex.
4. Returns `None` if all parsing attempts fail, triggering a fallback plan.

10. References

10.1 Requirements Document

The project is built against the Software Requirements Specification (SRS) document, stored in `requirements.txt` at the project root. This document contains:

- **Domain Description:** Entities (User, Chat, Message, File, Embedding), relationships, glossary (SDSA, VectorDB, PVC, etc.).
- **Functional Requirements:** 15+ FR groups covering chat management, messaging, workspace operations, web search, Spotify, voice I/O, file transforms, general tasks, conversational AI, task planning, and orchestration.
- **Non-Functional Requirements:** Performance targets (<10s response), scalability constraints (single GPU), reliability (persistent volumes), maintainability (modular agents), security (SSRF prevention, input sanitization), and privacy.
- **Tech Stack Requirements:** Frontend (Next.js SSR), backend (Python FastAPI), databases (MySQL, MinIO, ChromaDB), Kubernetes.
- **Database Schema:** Full table definitions, indices, constraints.
- **API Requirements:** Complete REST endpoint contracts for all agents and services.
- **Kubernetes Requirements:** Resource allocations, GPU scheduling, persistent storage.

10.2 Technology Documentation

Resource	URL
FastAPI Documentation	https://fastapi.tiangolo.com/
Next.js Documentation	https://nextjs.org/docs
Ollama Documentation	https://ollama.com/
ChromaDB Documentation	https://docs.trychroma.com/
MinIO Documentation	https://min.io/docs
Sentence-Transformers	https://www.sbert.net/
CLIP (OpenAI)	https://openai.com/research/clip
Faster Whisper	https://github.com/SYSTRAN/faster-whisper
Kubernetes Persistent Volumes	https://kubernetes.io/docs/concepts/storage/persistent-volumes/

Resource	URL
Kubernetes GPU Scheduling	https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/
DuckDuckGo (Lite)	https://html.duckduckgo.com/html/
Spotify Web API	https://developer.spotify.com/documentation/web-api/
Tesseract OCR	https://github.com/tesseract-ocr/tesseract
SQLAlchemy 2.0	https://docs.sqlalchemy.org/
Docker Compose	https://docs.docker.com/compose/
TailwindCSS	https://tailwindcss.com/docs

10.3 Academic / Conceptual References

- Microservices Architecture Patterns: HTTP is synchronous request/response — appropriate for sequential GPU-bound workflows.
 - RAG (Retrieval-Augmented Generation): Combining vector similarity search with LLM context injection.
 - SSR (Server-Side Rendering): Next.js SSR delivers content 30–50% faster than client-side rendering for initial page loads.
-

11. Appendix: Requirements Traceability

This section maps the requirements from the SRS to the implementation artifacts.

11.1 Functional Requirements Traceability

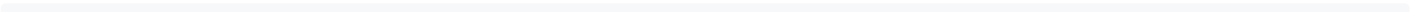
FR ID	Requirement	Implementing Component	Endpoint(s)
FR-1	List all chat sessions	ChatStore Agent → ChatMessages Svc	GET /chats
FR-2	Create a new chat	ChatStore Agent → ChatMessages Svc	POST /chats
FR-3	Delete a chat	ChatStore Agent → ChatMessages Svc	DELETE /chats/{id}
FR-4	Retrieve last N messages	ChatStore Agent → ChatMessages Svc	GET /chats/{id}/messages?limit=N
FR-5	Add user message	ChatStore Agent → ChatMessages Svc	POST /chats/{id}/messages
FR-6	Add system response	Orchestrator → ChatStore Agent	POST /chats/{id}/messages (role=system)
FR-7	Task planning	TaskPlanner Agent → Ollama	POST /plan
FR-8	Plan execution	Orchestrator Agent	POST /chat (internal loop)
FR-9	List workspace files	Workspace Agent → FileSystem Svc	GET /list?dir=...
FR-10	Upload files	Workspace Agent → Documents Svc + FileSystem Svc	POST /upload

FR-11 FR ID	Read/download files Requirement	Workspace Agent → Documents Svc Implementing Component	GET /file?path=... Endpoint(s)
FR-12	Copy files	Workspace Agent	POST /copy
FR-13	Move files	Workspace Agent	POST /move
FR-14	Delete files	Workspace Agent	DELETE /files
FR-15	Rename files	Workspace Agent	POST /rename
FR-16	Re-index on file change	Workspace Agent → SDSA Agent	Automatic notification
FR-17	Web search	WebSearch Agent	GET /search
FR-18	URL fetch & content extraction	WebSearch Agent	POST /fetch
FR-19	Spotify API key storage	APIKeys Service	PUT /keys/ {user_id}/spotify
FR-20	Music playback	Spotify Agent → Spotify Web API	POST /playback/ play, etc.
FR-21	Speech-to-text	VoicelO Agent (Whisper)	POST /stt
FR-22	Text-to-speech	VoicelO Agent (pyttsx3)	POST /tts
FR-23	File format conversion	FileTransform Agent	POST /convert
FR-24	Code execution	GeneralTask Agent	POST /code/execute
FR-25	LLM chat response	Conversational Agent → Ollama + SDSA	POST /chat

11.2 Non-Functional Requirements Traceability

NFR	Implementation
Responsiveness (<10s)	FastAPI async, sequential GPU calls, model caching
Persistent data	PVCs for MySQL, MinIO, ChromaDB, Ollama in K8s manifests
GPU constraint (1× RTX 3070)	nvidia.com/gpu: 1 in K8s; lazy load/unload pattern; sequential execution
Modularity	Independent FastAPI apps per agent/service; shared models via backend/shared/
Security (SSRF)	URL validation in WebSearch Agent; no private IPs in outbound requests
No external data leakage	All LLM inference runs locally via Ollama

11.3 Project Structure Map



```

my-pai/
├── DOCUMENTATION.md          # This file
├── README.md                 # Quick start guide
├── requirements.txt           # Software requirements specification
├── docker-compose.yaml       # Local development compose
├── commands.txt              # Useful commands
├── LICENSE                   # Project license
├──
├── backend/
│   ├── config.yaml           # Conda environment definition
│   ├── requirements.txt       # Full Python dependencies (agents)
│   ├── requirements-services.txt # Minimal Python dependencies (services)
│   ├──
│   ├── shared/               # Shared code across all components
│   │   ├── __init__.py
│   │   ├── config.py         # Centralized settings + URLs
│   │   ├── models.py         # Shared Pydantic models
│   │   └── utils.py          # Utility functions
│   ├──
│   ├── services/             # Data-layer microservices
│   │   ├── filesystem_service/ # Port 8001 – MySQL file metadata
│   │   ├── documents_service/  # Port 8002 – MinIO file content
│   │   ├── chatmessages_service/ # Port 8003 – MySQL chats & messages
│   │   ├── apikeys_service/    # Port 8004 – MySQL API keys
│   │   └── vectordb_service/    # Port 8005 – ChromaDB embeddings
│   ├──
│   ├── agents/               # Application-layer agents
│   │   ├── workspace_agent/    # Port 8010 – File operations
│   │   ├── chatstore_agent/    # Port 8011 – Chat management
│   │   ├── sdsa_agent/         # Port 8012 – Semantic search + embeddings
│   │   ├── websearch_agent/    # Port 8013 – Web search
│   │   ├── spotify_agent/      # Port 8014 – Spotify integration
│   │   ├── voiceio_agent/      # Port 8015 – Speech I/O
│   │   ├── filetransform_agent/ # Port 8016 – Format conversions
│   │   ├── generaltask_agent/   # Port 8017 – Math, code, analysis
│   │   ├── conversational_agent/ # Port 8018 – LLM chat + RAG
│   │   ├── taskplanner_agent/  # Port 8019 – Intent analysis + planning
│   │   └── orchestrator_agent/ # Port 8020 – Central coordinator
│   ├──
│   └── examples/             # Demo/test applications
│       ├── homework_example/
│       ├── multi_modal_embedding_example/
│       ├── Prompt-Engineering-Guide-main/
│       └── simple_example/
├──
├── frontend/                 # Next.js application
│   ├── package.json
│   ├── next.config.js
│   ├── tailwind.config.js
│   ├── tsconfig.json
│   ├── public/               # Static assets
│   └── src/
│       ├── pages/            # Route pages
│       │   ├── index.tsx     # Home
│       │   ├── _app.tsx      # App wrapper
│       │   ├── _document.tsx # HTML document
│       │   ├── chat/         # Chat interface pages
│       │   ├── workspace/    # Workspace browser pages
│       │   └── settings/     # Settings pages
│       ├── components/       # React components
│       │   ├── Chat/         # ChatList, ChatMessage, ChatInput
│       │   ├── Workspace/    # FileBrowser, FilePreview
│       │   └── Layout/       # Sidebar
│       ├── lib/              # Utility functions
│       └── styles/           # Global styles

```

```
|
├─ docker/                                # Dockerfiles
│   ├── Dockerfile.agents                 # Multi-stage agent builds (CUDA base)
│   ├── Dockerfile.services              # Multi-stage service builds (slim base)
│   └─ Dockerfile.frontend                # Multi-stage Next.js build
├─ kubernetes/                           # K8s manifests
│   ├── 01-infrastructure.yaml            # Namespace, ConfigMap, Secrets, PVCs, DBs
│   ├── 02-services.yaml                 # Data-layer service deployments
│   ├── 03-agents.yaml                   # Agent deployments (incl. GPU specs)
│   └─ 04-frontend.yaml                  # Frontend deployment + Ingress
└─ static/
    ├── initial_prompt.txt                # SRS generation prompt
    └─ Project Description/               # Project description assets
```